Plan de desarrollo — Aplicación móvil de préstamos

Versión: 2.0
Fecha: 1 de noviembre de 2026
Estado: Plan de implementación
Base: Documento maestro de la aplicación móvil de préstamos (v1.0)

1. Propósito de este documento

Este documento convierte la especificación funcional original en un plan práctico para construir la aplicación por bloques. Mantiene el alcance y las reglas funcionales del documento maestro, pero simplifica la estructura técnica y establece una secuencia clara de implementación.

El proyecto ya cuenta con:

- Proyecto React Native con Expo y TypeScript.
- Conexión con Supabase.
- Base de datos PostgreSQL creada.
- Tablas creadas.

Por tanto, el trabajo no comienza desde cero. El primer bloque consiste en revisar que lo existente soporte correctamente los flujos definidos y completar únicamente lo que falte.

Objetivo de la aplicación

Construir una aplicación móvil privada para que un prestamista pueda administrar:

- Personas a las que les presta dinero.
- Préstamos y sus condiciones.
- Cuotas y vencimientos.
- Pagos, abonos y distribución del dinero.
- Intereses, saldos y atrasos.
- Historial de movimientos.
- Indicadores del negocio.

En la primera versión habrá un usuario prestamista autenticado. Las personas prestatarias serán registros internos y no tendrán acceso a la aplicación.

Fuera del alcance de la primera versión

- Aplicación o portal para prestatarios.
- Integraciones bancarias o pasarelas de pago.
- Cobros automáticos.
- Mensajería masiva.
- Firma electrónica.
- Multiempresa o roles complejos.
- Contabilidad formal completa.

2. Decisiones técnicas y principios

Área	Decisión
Aplicación móvil	React Native + Expo

Lenguaje	TypeScript
Navegación	Expo Router
Estilos	NativeWind
Backend y persistencia	Supabase + PostgreSQL
Autenticación	Supabase Auth
Seguridad de datos	PostgreSQL RLS
Animaciones	React Native Reanimated
Organización	Arquitectura modular sencilla por funcionalidad

Principios de implementación

1. Organizar el código por funcionalidades, no crear una arquitectura empresarial con muchas capas.
2. Mantener la lógica de una funcionalidad dentro de su módulo.
3. Compartir únicamente componentes, utilidades y configuración que realmente se reutilicen.
4. Evitar consultas directas a Supabase desde los componentes visuales.
5. Mantener los cálculos financieros y las operaciones críticas en funciones controladas por PostgreSQL.
6. No duplicar reglas de negocio entre pantallas.
7. No introducir abstracciones hasta que exista una necesidad real.
8. Implementar y probar un bloque antes de avanzar al siguiente.

---

3. Estructura del proyecto propuesta

La estructura debe ser pequeña, fácil de entender y suficiente para el alcance actual.

```
src/
├── app/                      # Rutas y layouts de Expo Router
│   ├── (auth)/
│   │   ├── login.tsx
│   │   └── recuperar-password.tsx
│   ├── (tabs)/
│   │   ├── index.tsx         # Dashboard
│   │   ├── personas.tsx
│   │   ├── prestamos.tsx
│   │   └── configuracion.tsx
│   ├── personas/
│   │   ├── crear.tsx
│   │   └── [id].tsx
│   ├── prestamos/
│   │   ├── crear.tsx
│   │   └── [id].tsx
│   ├── pagos/
│   │   ├── registrar.tsx
│   │   └── [id].tsx
│   └── _layout.tsx
│
├── modules/
│   ├── auth/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types.ts
│   ├── people/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types.ts
│   ├── loans/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types.ts
│   ├── payments/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types.ts
│   ├── dashboard/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── services/
│   └── settings/
│       ├── components/
│       ├── hooks/
│       └── services/
│
├── shared/
│   ├── components/
│   ├── hooks/
│   ├── utils/
│   ├── constants/
│   └── types/
│
└── lib/
    └── supabase.ts
```

> La estructura es una guía, no una obligación de crear todos los archivos y carpetas desde el primer día. Crear únicamente lo que una funcionalidad necesite.

Responsabilidad de cada área

- **app:** contiene las rutas de Expo Router. Las pantallas conectan la navegación con los módulos.
- **modules:** contiene las funcionalidades del negocio. Cada módulo organiza sus componentes, hooks, servicios y tipos.
- **shared:** contiene elementos reutilizables por dos o más módulos, como botones, campos, formatos y estados comunes.

- **lib:** contiene la inicialización de herramientas externas, como el cliente de Supabase.

Reglas para evitar acoplamiento

- Un componente de interfaz no debe contener consultas SQL ni cálculos financieros.
- Las pantallas deben utilizar hooks o funciones del módulo correspondiente.
- Los servicios de cada módulo concentran el acceso a datos de esa funcionalidad.
- Un módulo no debe importar archivos internos de otro módulo sin una necesidad clara. Si se comparte una pieza, debe exponerse mediante una interfaz sencilla.
- No crear módulos independientes para cuotas, movimientos o auditoría si, en esta etapa, su lógica puede mantenerse dentro de `loans` o `payments`. Se pueden separar más adelante si el tamaño lo justifica.

---

4. Estado actual y trabajo pendiente de infraestructura

La conexión y las tablas ya existen. No se debe recrear el proyecto de Supabase ni ejecutar nuevamente scripts de creación sin revisar primero el estado actual.

Revisión inicial obligatoria

Elemento	Acción
Cliente Supabase	Verificar que la conexión funcione desde la aplicación.
Variables de entorno	Verificar que las claves públicas estén configuradas correctamente y no estén expuestas claves privilegiadas.
Tablas	Comparar las tablas existentes con las entidades y relaciones necesarias.
Relaciones	Comprobar claves foráneas, restricciones e integridad entre propietario, persona, préstamo y pago.
RLS	Verificar que esté habilitado en todas las tablas expuestas y probar el aislamiento por usuario.
Funciones SQL/RPC	Identificar cuáles existen y cuáles hacen falta para operaciones financieras.
Tipos TypeScript	Generar o actualizar los tipos de la base de datos, si aplica.

Tablas y entidades contempladas en el documento maestro

- `profiles`
- `people`
- `loan_settings`
- `loans`
- `installments`
- `payments`
- `payment_allocations`
- `loan_movements`

La lista anterior es la referencia funcional del documento maestro. **No significa que deban crearse de nuevo**. Se debe comparar con el esquema real que ya está implementado y adaptar el plan a los nombres y relaciones existentes.

Resultado esperado

Al terminar la revisión debe existir un registro breve con:

- Tablas existentes y su propósito.
- Relaciones y restricciones verificadas.

- Estado de RLS por tabla.
- Políticas faltantes o incorrectas.
- Funciones RPC existentes y pendientes.
- Diferencias entre el esquema actual y los flujos de la aplicación.

No avanzar a operaciones financieras reales mientras la propiedad de los registros y las políticas de seguridad no estén verificadas.

---

5. Reglas de negocio que deben respetarse

Esta sección conserva las reglas del documento maestro. Los puntos marcados como pendientes son bloqueantes para implementar los cálculos correspondientes.

Reglas conocidas

1. La tasa ordinaria predeterminada propuesta es del 20 % mensual.
2. Debe poder configurarse una persona exenta de interés ordinario.
3. Cada préstamo debe conservar una copia de las condiciones pactadas al crearse. Cambiar la tasa predeterminada no debe modificar préstamos anteriores.
4. El capital original, el capital pendiente, los intereses generados y los intereses cobrados son conceptos diferentes.
5. Solo se contabiliza como ganancia cobrada el interés efectivamente recibido y asignado a un pago confirmado.
6. Los importes deben manejarse con tipos numéricos adecuados en PostgreSQL y con reglas explícitas de redondeo.
7. Las correcciones financieras deben conservar el movimiento original y registrar reversos o ajustes auditables.

Decisiones pendientes que bloquean cálculos

ID	Decisión	Qué se debe confirmar
DP-01	Fórmula de interés ordinario	Base de cálculo, periodicidad y momento en que se genera.
DP-02	Exención	Si aplica globalmente a la persona o se define por préstamo; confirmar si afecta la mora.
DP-03	Modalidad	Cuotas mensuales, interés periódico con capital al final u otra modalidad.
DP-04	Mora	Fórmula, base, periodicidad, capitalización y tratamiento de abonos.
DP-05	Orden de pago	Si el pago cubre primero mora, interés o capital.
DP-06	Pagos parciales	Confirmar si se permiten y cómo se distribuyen.
DP-07	Excedentes	Definir qué ocurre si el pago supera la deuda pendiente.
DP-08	Redondeo	Definir precisión y regla de redondeo para los cálculos.

El ejemplo de $100.000 que pasa a $144.000 no determina por sí solo una fórmula inequívoca. No programar la mora ni la capitalización basándose únicamente en ese ejemplo.

La tasa y los cargos deben validarse conforme a la normativa colombiana aplicable antes de utilizarlos en producción.

---

6. Plan de desarrollo por bloques

El desarrollo se divide en ocho bloques. Cada uno tiene un objetivo, funcionalidades, requisitos relacionados, dependencias y criterios de aceptación.

Bloque 0 — Revisión del proyecto existente

Objetivo: conocer y validar lo que ya está construido antes de agregar funcionalidades.

Incluye:

- Revisar la estructura actual de Expo.
- Verificar navegación y configuración de TypeScript.
- Comprobar conexión con Supabase.
- Revisar las tablas existentes y compararlas con el modelo funcional.
- Verificar relaciones, restricciones y tipos.
- Auditar las políticas RLS existentes.
- Identificar funciones RPC disponibles y pendientes.
- Revisar variables de entorno y confirmar que no haya claves privilegiadas en la aplicación.

Requisitos relacionados: RNF-001, RNF-003, RNF-006 y RNF-010.

Flujo relacionado: preparación técnica; todavía no es un flujo de negocio para el usuario.

Criterios para cerrar el bloque:

- La aplicación se conecta correctamente a Supabase.
- Se conoce el esquema real de la base de datos.
- Se documentaron las diferencias entre el esquema actual y el modelo requerido.
- Se identificaron las políticas RLS que faltan.
- No se recrearon tablas que ya existían.
- Se dispone de una lista concreta de ajustes pendientes.

No avanzar si: no se puede demostrar que un usuario autenticado solo accede a sus propios registros.

Bloque 1 — Fundamentos de la aplicación y autenticación

Objetivo: dejar lista la navegación principal y permitir el acceso seguro del prestamista.

Requisitos funcionales:

- RF-001: iniciar y cerrar sesión.
- RF-014: estados de carga, error, éxito y reintento.

Pantallas:

- SCR-01 Splash / carga.
- SCR-02 Inicio de sesión.
- SCR-03 Recuperar contraseña.
- SCR-22 Estados comunes.

Flujo FL-01 — Inicio de sesión:

1. El usuario abre la aplicación.
2. La aplicación comprueba si existe una sesión válida.
3. Si no existe sesión, muestra el inicio de sesión.
4. El usuario introduce sus credenciales.
5. Se validan los campos.
6. Supabase Auth autentica al usuario.
7. Si la autenticación es correcta, se dirige al Dashboard.

8. Si falla, se muestra un mensaje claro sin revelar información sensible.

Flujo complementario — Cerrar sesión:

1. El usuario abre la configuración o perfil.
2. Selecciona cerrar sesión.
3. La sesión se cierra mediante Supabase Auth.
4. La aplicación limpia el estado privado en memoria.
5. Se redirige al inicio de sesión.

Criterios de aceptación:

- El usuario puede iniciar y cerrar sesión.
- Una sesión válida se conserva según la configuración de Auth.
- Un usuario sin sesión no puede entrar a rutas privadas.
- Los errores de autenticación se muestran correctamente.
- Recuperar contraseña presenta estados de envío y confirmación.
- No se muestran datos privados mientras se inicializa o valida la sesión.

Bloque 2 — Personas y configuración básica

Objetivo: administrar las personas a las que se les presta dinero y las condiciones predeterminadas para nuevos préstamos.

Requisitos funcionales:

- RF-002: crear, consultar y editar personas.
- RF-003: configurar exención de interés ordinario.
- RF-005: tasa predeterminada del 20 %, pendiente de aprobación y validación.
- RF-012: administrar configuración de intereses.
- RF-013: filtros por nombre y otros criterios aplicables.

Pantallas:

- SCR-05 Personas.
- SCR-06 Crear/editar persona.
- SCR-07 Detalle de persona.
- SCR-19 Configuración de intereses.
- SCR-21 Perfil y preferencias, solo si se requiere en este bloque.

Flujo FL-02 — Crear persona:

1. El usuario entra a Personas.
2. Selecciona crear persona.
3. Introduce nombre y, opcionalmente, teléfono y notas.
4. Define si la persona tiene exención de interés ordinario, según la decisión aprobada.
5. La aplicación valida los campos.
6. El servicio del módulo envía la operación a Supabase.
7. La base de datos valida la propiedad del usuario mediante RLS.
8. Se confirma la creación y se abre el detalle de la persona.

Flujo — Editar persona:

1. El usuario abre el detalle de una persona.
2. Selecciona editar.
3. Modifica los datos permitidos.
4. Guarda los cambios.
5. La aplicación confirma el resultado y actualiza la información.

Flujo — Configurar tasa predeterminada:

1. El usuario abre Configuración de intereses.
2. Consulta la tasa predeterminada actual.
3. Introduce una nueva tasa.
4. La aplicación valida el valor.
5. Se guarda la configuración del usuario.
6. La nueva tasa se utiliza únicamente en préstamos creados posteriormente.

Criterios de aceptación:

- Se puede crear, consultar y editar una persona.
- Los campos opcionales pueden quedar vacíos.
- No se puede crear una persona sin nombre válido.
- El usuario no puede consultar ni modificar personas de otro propietario.
- La exención queda almacenada y se aplica según el alcance aprobado.
- Cambiar la tasa predeterminada no modifica préstamos existentes.
- Los listados muestran carga, error y estado vacío.

Bloque 3 — Préstamos y cuotas

Objetivo: registrar préstamos, congelar sus condiciones y representar sus obligaciones y vencimientos.

Requisitos funcionales:

- RF-004: crear préstamos con persona, capital, fecha, vencimiento y condiciones.
- RF-005: aplicar la tasa predeterminada aprobada.
- RF-006: representar cuotas y obligaciones.
- RF-010: identificar obligaciones vencidas, sin aplicar cargos no aprobados.
- RF-013: filtrar préstamos.

Pantallas:

- SCR-08 Préstamos.
- SCR-09 Crear préstamo.
- SCR-10 Detalle de préstamo.
- SCR-11 Cuotas y vencimientos.
- SCR-12 Detalle de cuota.

Flujo FL-03 — Crear préstamo:

1. El usuario entra a Préstamos.
2. Selecciona crear préstamo.
3. Busca y selecciona una persona existente.
4. Introduce el capital.
5. Define fecha de desembolso y vencimiento o modalidad de cuotas.
6. La aplicación carga la tasa predeterminada y permite establecer las condiciones permitidas.
7. Se determina si aplica exención de interés.
8. Se muestra un resumen de las condiciones.
9. El usuario confirma.
10. El servidor valida propietario, persona, importes y condiciones.
11. Se crea el préstamo y las cuotas u obligaciones correspondientes mediante una operación consistente.
12. Se muestra el detalle del préstamo.

Flujo — Consultar préstamo y cuotas:

1. El usuario abre el listado de préstamos.

2. Puede filtrar por persona, estado o fecha.
3. Selecciona un préstamo.
4. Consulta capital original, saldo, intereses, condiciones y vencimientos.
5. Puede abrir el detalle de una cuota para consultar importe, fecha, estado y pagos asociados.

Criterios de aceptación:

- No se puede crear un préstamo sin persona válida.
- El capital debe ser mayor que cero.
- Las fechas deben respetar las reglas aprobadas.
- Las condiciones quedan guardadas como snapshot del préstamo.
- Las cuotas u obligaciones corresponden a la modalidad aprobada.
- Un préstamo asociado a una persona de otro usuario debe ser rechazado.
- Los saldos mostrados distinguen capital e intereses.
- Los cálculos coinciden con casos de referencia aprobados.

Dependencia crítica: no implementar la generación definitiva de intereses ni cuotas hasta aprobar periodicidad, fórmula y modalidad.

Bloque 4 — Pagos, abonos e historial financiero

Objetivo: registrar el dinero recibido, distribuirlo correctamente y conservar la trazabilidad de cada operación.

Requisitos funcionales:

- RF-007: registrar pagos de capital, intereses o distribución mixta.
- RF-008: registrar asignaciones del pago.
- RF-009: consultar historial financiero.
- RF-015: conservar auditoría de pagos y ajustes.
- RF-006: permitir pagos parciales, si la regla es aprobada.
- RF-014: estados de operación y reintento seguro.

Pantallas:

- SCR-10 Detalle de préstamo.
- SCR-12 Detalle de cuota.
- SCR-13 Registrar pago.
- SCR-14 Confirmación de pago.
- SCR-15 Historial de pagos.
- SCR-16 Movimiento financiero.

Flujo FL-04 — Registrar pago:

1. El usuario abre un préstamo o una cuota.
2. Selecciona registrar pago.
3. Introduce importe y fecha de recepción.
4. Selecciona el tipo de pago o consulta la distribución propuesta por el sistema.
5. El servidor calcula el saldo vigente y valida el importe.
6. La aplicación presenta una previsualización de la distribución.
7. El usuario revisa y confirma.
8. La aplicación envía la operación con una clave de idempotencia.
9. PostgreSQL valida sesión, propiedad, saldo y reglas.
10. En una sola transacción se registra el pago, sus asignaciones y movimientos.
11. Se devuelve el resultado confirmado.
12. La aplicación muestra el comprobante o confirmación y actualiza el historial y los saldos.

Flujo FL-05 — Pago parcial:

1. El usuario selecciona registrar un abono.
2. Introduce un importe menor que el saldo exigible, si está permitido.
3. El servidor valida el importe y calcula la distribución conforme al orden aprobado.
4. Se registra el pago y sus asignaciones transaccionalmente.
5. Se recalculan los saldos.
6. La obligación continúa pendiente o atrasada si conserva saldo y corresponde según su vencimiento.

Flujo FL-06 — Pago total:

1. El usuario selecciona pagar el saldo total.
2. El servidor calcula el saldo exigible actualizado.
3. La aplicación presenta el desglose.
4. El usuario confirma.
5. El servidor registra el pago y las asignaciones.
6. Si no queda saldo exigible, la obligación se considera pagada.
7. Se actualizan el detalle y el historial.

Flujo FL-10 — Corregir operación:

1. El usuario autorizado identifica el movimiento incorrecto.
2. Selecciona la acción de corrección disponible.
3. El sistema muestra el movimiento original y el ajuste propuesto.
4. El usuario confirma.
5. El servidor registra un reverso o movimiento compensatorio vinculado al original.
6. Se recalculan los saldos.
7. El historial conserva tanto la operación original como la corrección.

Flujo FL-11 — Error de red o reintento:

1. El usuario confirma una operación.
2. La aplicación pierde la conexión o no recibe respuesta.
3. No asume que el pago falló.
4. Consulta el resultado mediante la clave de idempotencia.
5. Si ya fue registrado, muestra el resultado existente.
6. Si no fue registrado, permite reintentar de forma segura.

Criterios de aceptación:

- Un pago confirmado tiene asignaciones cuya suma coincide con el importe recibido.
- Un pago de intereses no reduce indebidamente el capital.
- No se duplica un pago al repetir la misma clave idempotente.
- Dos pagos simultáneos no pueden sobreasignar el saldo.
- No se permite modificar o eliminar silenciosamente un pago confirmado.
- Los reversos conservan la referencia a la operación original.
- Los errores de red no provocan registros duplicados.
- Los cálculos y la distribución se validan en el servidor.

Dependencia crítica: este bloque requiere decisiones aprobadas sobre pagos parciales, orden de aplicación, pagos excedentes y fórmula de intereses.

Bloque 5 — Mora, estados y vencimientos

Objetivo: mostrar obligaciones pendientes, atrasadas y pagadas, y aplicar la regla de mora únicamente cuando esté definida.

Requisitos funcionales:

- RF-010: detectar obligaciones vencidas y aplicar reglas confirmadas.
- RF-006: consultar cuotas y obligaciones.
- RF-013: filtrar por estado y fecha.
- RF-011: suministrar información de atrasos al Dashboard.

Pantallas:

- SCR-11 Cuotas y vencimientos.
- SCR-12 Detalle de cuota.
- SCR-17 Atrasados.
- SCR-18 Próximos vencimientos.
- SCR-20 Configuración de mora.

Flujo FL-07 — Mora:

1. El usuario consulta las obligaciones.
2. El sistema identifica las que tienen fecha vencida y saldo exigible.
3. Determina si la obligación cumple las condiciones para mora.
4. Si existe una regla aprobada, calcula el cargo en el servidor.
5. Registra el cargo de forma idempotente y auditable, si corresponde.
6. Actualiza el saldo y el estado mostrado.
7. Si la regla no está configurada o aprobada, muestra el atraso sin inventar ni aplicar un recargo.

Flujo — Consultar próximos vencimientos:

1. El usuario abre Próximos vencimientos o el acceso correspondiente desde Dashboard.
2. Selecciona o utiliza el periodo disponible.
3. El sistema lista obligaciones con saldo y vencimiento dentro del rango.
4. El usuario abre una obligación para consultar su detalle.

Estados funcionales

- **Pendiente:** aún no vence y conserva saldo.
- **Retrasado:** venció y conserva saldo exigible.
- **Pagado:** no conserva saldo exigible.
- **Pago a tiempo:** resultado/evento de un pago realizado dentro del vencimiento.
- **Pago de interés:** tipo de movimiento; no implica por sí mismo que el préstamo esté pagado.

El estado del préstamo, el estado de la cuota y el tipo de pago deben mantenerse como conceptos distintos.

Criterios de aceptación:

- Una obligación vencida con saldo aparece como atrasada.
- Una obligación sin saldo aparece como pagada.
- Un pago de interés no marca automáticamente el préstamo como pagado.
- La mora no se aplica si su fórmula no está aprobada.
- Reprocesar la detección de mora no duplica cargos.
- Los filtros por fecha y estado devuelven resultados coherentes.

Bloque 6 — Dashboard y métricas

Objetivo: ofrecer una vista general del estado del negocio y permitir navegar desde cada indicador hacia los registros que lo componen.

Requisitos funcionales:

- RF-011: métricas financieras y personas atrasadas.
- RF-013: filtros por periodo cuando corresponda.

- RF-014: estados de carga, error y vacío.

Pantallas:

- SCR-04 Dashboard.
- Accesos relacionados a SCR-05 Personas, SCR-08 Préstamos, SCR-17 Atrasados y SCR-18 Próximos vencimientos.

Flujo FL-08 — Dashboard:

1. El usuario inicia sesión y entra al Dashboard.
2. La aplicación consulta los agregados del usuario autenticado.
3. Presenta los indicadores disponibles.
4. Muestra próximos vencimientos y alertas relevantes.
5. El usuario selecciona una tarjeta o indicador.
6. La aplicación abre el listado correspondiente con el filtro aplicado.

Métricas que deben implementarse:

Indicador	Definición
Capital prestado histórico	Suma del capital original desembolsado, sin restar recuperaciones.
Capital pendiente	Capital desembolsado menos pagos asignados a capital y ajustes aplicables.
Ganancias cobradas	Intereses efectivamente recibidos, menos reversos.
Intereses pendientes	Intereses exigibles generados menos importes asignados a intereses.
Cartera pendiente	Capital, intereses y cargos pendientes según las condiciones del préstamo.
Personas atrasadas	Personas distintas con al menos una obligación vencida y saldo exigible.
Personas pendientes	Personas distintas con alguna obligación pendiente.
Pagos del periodo	Pagos confirmados recibidos dentro del periodo.
Próximos vencimientos	Obligaciones con saldo y vencimiento dentro del horizonte consultado.

Criterios de aceptación:

- Los indicadores se calculan solo con datos del usuario autenticado.
- Capital recuperado no se suma como ganancia.
- Los conteos de personas no duplican a alguien con varias cuotas atrasadas.
- Cada tarjeta navega al listado que explica su valor.
- Los valores se contrastan con un conjunto de datos de prueba calculado manualmente.
- Se muestran estados vacíos cuando todavía no existen préstamos o pagos.

Bloque 7 — Estabilización y entrega privada

Objetivo: revisar la aplicación completa, corregir defectos y preparar una versión estable para uso privado.

Incluye:

- Pruebas de flujos completos.
- Pruebas unitarias de cálculos y validaciones.
- Pruebas de integración con Supabase.
- Pruebas RLS con dos usuarios.

- Pruebas de idempotencia y concurrencia.
- Revisión de accesibilidad y rendimiento.
- Estados de carga, error, vacío y reintento.
- Animaciones sutiles con Reanimated.
- Respeto a la preferencia de reducir movimiento.
- Verificación en dispositivos Android y/o iOS objetivo.
- Revisión de backups y procedimiento de recuperación.
- Configuración de distribución privada.

Requisitos relacionados: RNF-001 a RNF-010 y RF-014, RF-015.

Criterios de aceptación:

- Todos los flujos prioritarios funcionan de principio a fin.
- No existen defectos críticos conocidos.
- Las pruebas de aislamiento RLS pasan.
- Los pagos no se duplican ni generan saldos inconsistentes.
- Las métricas coinciden con los resultados de referencia.
- Se ha probado la recuperación de datos según el procedimiento definido.
- La aplicación se ha probado en los dispositivos objetivo.

---

7. Matriz de requisitos funcionales y flujos

Esta matriz permite comprobar que ningún requisito funcional queda sin un flujo de implementación.

Requisito	Descripción	Flujo principal	Bloque
RF-001	Autenticación	FL-01 Inicio de sesión	1
RF-002	Gestión de personas	FL-02 Crear/editar persona	2
RF-003	Exención de interés	FL-02 Crear/editar persona	2
RF-004	Crear préstamo	FL-03 Crear préstamo	3
RF-005	Tasa predeterminada	Configurar tasa / FL-03	2 y 3
RF-006	Cuotas y obligaciones	Consultar cuotas / FL-05 / FL-06	3, 4 y 5
RF-007	Registrar pagos	FL-04 Registrar pago	4
RF-008	Asignación de pagos	FL-04 / FL-05 / FL-06	4
RF-009	Historial financiero	Consultar historial / FL-10	4
RF-010	Mora	FL-07 Mora	3 y 5
RF-011	Dashboard	FL-08 Dashboard	6
RF-012	Configuración	Configurar tasa / configurar mora	2 y 5
RF-013	Filtros	Listados de personas, préstamos y obligaciones	2, 3 y 5
RF-014	Estados y errores	Todos los flujos	1 a 7
RF-015	Auditoría	FL-04 y FL-10	4

Requisitos no funcionales y bloques de verificación

Requisito	Verificación principal

RNF-001 Seguridad	Bloques 0, 1 y 7: RLS y pruebas con dos usuarios.
RNF-002 Consistencia financiera	Bloques 3, 4 y 7: transacciones y concurrencia.
RNF-003 Privacidad	Bloques 0, 1 y 7: variables, sesión y datos mínimos.
RNF-004 Usabilidad	Todos los bloques: formularios, validaciones y errores.
RNF-005 Accesibilidad	Bloques 1, 2 y 7.
RNF-006 Mantenibilidad	Bloques 0 y 7: estructura sencilla y lógica fuera de componentes.
RNF-007 Rendimiento	Bloques 2, 3, 6 y 7: listas y consultas eficientes.
RNF-008 Recuperación	Bloque 7: backups y restauración.
RNF-009 Compatibilidad	Bloque 7: dispositivos objetivo.
RNF-010 Trazabilidad	Bloques 4 y 7: historial, reversos y auditoría.

8. Mapa de navegación funcional



9. Inventario de pantallas por bloque

Pantalla	Nombre	Bloque
SCR-01	Splash / carga	1
SCR-02	Inicio de sesión	1
SCR-03	Recuperar contraseña	1
SCR-04	Dashboard	6

SCR-05	Personas	2
SCR-06	Crear/editar persona	2
SCR-07	Detalle de persona	2
SCR-08	Préstamos	3
SCR-09	Crear préstamo	3
SCR-10	Detalle de préstamo	3 y 4
SCR-11	Cuotas y vencimientos	3 y 5
SCR-12	Detalle de cuota	3, 4 y 5
SCR-13	Registrar pago	4
SCR-14	Confirmación de pago	4
SCR-15	Historial de pagos	4
SCR-16	Movimiento financiero	4
SCR-17	Atrasados	5
SCR-18	Próximos vencimientos	5
SCR-19	Configuración de intereses	2
SCR-20	Configuración de mora	5
SCR-21	Perfil y preferencias	1 o 7
SCR-22	Estados comunes	Todos

Las pantallas son unidades funcionales, no necesariamente archivos independientes. Se pueden reutilizar componentes y formularios cuando los comportamientos sean equivalentes.

10. Estrategia de pruebas

Las pruebas se implementan junto con cada bloque; el bloque 7 concentra la regresión completa.

Casos funcionales y financieros

ID	Escenario	Resultado esperado
T-001	Préstamo de $100.000 con tasa ordinaria aprobada	Resultado coincide con fórmula y redondeo aprobados.
T-002	Persona exenta de interés ordinario	Interés ordinario cero, con tratamiento de mora según regla aprobada.
T-003	Obligación vencida con saldo	Estado atrasado; recargo solo si está aprobado.
T-004	Pago de solo intereses	Disminuye interés pendiente; no reduce capital indebidamente.
T-005	Pago mixto	Las asignaciones suman exactamente el importe recibido.
T-006	Misma clave idempotente repetida	No crea un segundo pago.
T-007	Dos pagos simultáneos sobre el mismo saldo	No hay sobreasignación ni cargos duplicados.
T-008	Usuario A intenta acceder a registro de B	Acceso denegado.
T-009	Pago mayor que la deuda	Se aplica la política aprobada sin saldo negativo accidental.

T-010	Reversión de pago	Se conserva el original y se registra el movimiento compensatorio.
T-011	Cambio de tasa predeterminada	Solo afecta nuevos préstamos.
T-012	Dashboard sin registros	Muestra estados vacíos útiles.

Pruebas por bloque

- **Bloque 0:** conexión, esquema, relaciones y RLS.
- **Bloque 1:** sesión, rutas protegidas y errores de autenticación.
- **Bloque 2:** validaciones y aislamiento de personas/configuración.
- **Bloque 3:** creación de préstamos, condiciones y obligaciones.
- **Bloque 4:** asignaciones, transacciones, idempotencia, concurrencia y reversos.
- **Bloque 5:** vencimientos, estados y aplicación controlada de mora.
- **Bloque 6:** exactitud de métricas y navegación desde indicadores.
- **Bloque 7:** regresión completa, dispositivos y recuperación.

---

11. Forma de trabajo y control de avance

Cada bloque debe ejecutarse siguiendo este ciclo:

1. **Analizar:** revisar requisitos, pantallas, dependencias y tablas afectadas.
2. **Planificar:** dividir el bloque en tareas pequeñas y ordenadas.
3. **Implementar:** desarrollar la funcionalidad sin adelantar bloques dependientes.
4. **Probar:** ejecutar los casos de aceptación y corregir fallos.
5. **Revisar:** comprobar que el código respeta la estructura modular y las reglas de negocio.
6. **Cerrar:** documentar qué se completó, qué cambió y qué queda pendiente.

Regla de cierre

Un bloque no se considera terminado solo porque la pantalla se vea o porque la operación funcione en un caso sencillo. Debe cumplir sus criterios de aceptación y pruebas.

Registro recomendado por bloque

```
Bloque:
Objetivo:

Tareas completadas:
- [ ] ...

Requisitos cubiertos:
- RF-...

Flujos implementados:
- FL-...

Pantallas implementadas:
- SCR-...

Tablas / funciones afectadas:
- ...

Pruebas ejecutadas:
- ...

Errores encontrados:
- ...

Pendientes:
- ...

Criterio de salida:
- [ ] Cumplido

Estado: Pendiente / En progreso / En revisión / Completado
```

Reglas para el agente de desarrollo

Si este documento se utiliza como contexto para un agente de programación:

- Leer el documento completo antes de modificar el proyecto.
- Inspeccionar la estructura y el código existentes antes de crear archivos.
- No reemplazar ni recrear la conexión de Supabase.
- No recrear tablas que ya existan.
- No cambiar nombres de tablas o columnas sin justificarlo y revisar sus dependencias.
- Trabajar únicamente en el bloque solicitado.
- No implementar fórmulas financieras que estén marcadas como pendientes.
- No asumir que una validación de interfaz sustituye RLS o validaciones del servidor.
- Mantener los componentes visuales separados de las consultas y reglas financieras.
- Ejecutar pruebas antes de declarar una tarea terminada.
- Informar los archivos modificados, decisiones tomadas, pruebas ejecutadas y pendientes.

---

12. Orden recomendado de ejecución

Orden	Bloque	Resultado principal
0	Revisión del proyecto existente	Conexión, esquema y seguridad conocidos y verificados.
1	Fundamentos y autenticación	Acceso seguro y navegación base.
2	Personas y configuración	Personas administrables y condiciones predeterminadas.
3	Préstamos y cuotas	Préstamos creados con condiciones y obligaciones.
4	Pagos e historial	Registro financiero consistente y

		trazable.
5	Mora y vencimientos	Estados y atrasos confiables.
6	Dashboard	Métricas y navegación contextual.
7	Estabilización y entrega	Aplicación probada y lista para uso privado.

Los bloques 3, 4 y 5 dependen de decisiones financieras. Se puede avanzar en pantallas, estructura y consultas no ambiguas, pero no activar cálculos definitivos hasta aprobar las reglas correspondientes.

---

13. Lista de decisiones antes de implementar operaciones financieras

- [ ] Confirmar fórmula y periodicidad del interés ordinario.
- [ ] Confirmar si la exención es por persona o por préstamo.
- [ ] Confirmar si la exención también aplica a mora.
- [ ] Confirmar modalidad de devolución y generación de cuotas.
- [ ] Confirmar fórmula y periodicidad de mora.
- [ ] Confirmar orden de aplicación de pagos.
- [ ] Confirmar si se permiten pagos parciales.
- [ ] Definir tratamiento de pagos superiores al saldo.
- [ ] Definir redondeo y precisión.
- [ ] Validar legalmente las tasas y cargos aplicables.
- [ ] Verificar que el esquema actual y sus políticas RLS soportan los flujos.
- [ ] Definir y probar las funciones transaccionales necesarias.

---

14. Resultado esperado al finalizar la primera versión

La aplicación debe permitir al prestamista:

1. Iniciar y cerrar sesión de forma segura.
2. Crear, consultar y editar personas.
3. Configurar exenciones y tasa predeterminada.
4. Crear préstamos asociados a personas.
5. Consultar condiciones, cuotas y vencimientos.
6. Registrar pagos y abonos según reglas aprobadas.
7. Consultar la distribución de cada pago.
8. Consultar historial y movimientos financieros.
9. Identificar obligaciones pendientes y atrasadas.
10. Consultar métricas de capital, cartera, intereses y personas.
11. Corregir operaciones mediante ajustes auditables.
12. Mantener los datos aislados por usuario y las operaciones financieras consistentes.

Criterio final: la primera versión se considera lista cuando los flujos principales funcionan de principio a fin, los cálculos financieros han sido aprobados y probados, RLS impide el acceso cruzado y los movimientos quedan registrados sin duplicaciones ni pérdida de trazabilidad.

---

15. Referencia al documento maestro

Este plan deriva del documento **“Documento maestro — Aplicación móvil de préstamos”**, versión 1.0 del 23 de octubre de 2026.

Se conservan su alcance, actores, requisitos RF-001 a RF-015, requisitos no funcionales RNF-001 a RNF-010, flujos FL-01 a FL-11, pantallas SCR-01 a SCR-22, conceptos de datos, controles RLS, integridad financiera, métricas y estrategia de pruebas.

Este documento reorganiza el plan de implementación para hacerlo ejecutable sobre el proyecto ya iniciado. Cuando exista una diferencia entre una implementación actual y un requisito de negocio, se debe documentar y validar antes de modificar datos o comportamiento.